

Razvoj web aplikacija - Predavanje 4

Autorizacija, role i ownership

mag.ing. Josip Torić (SIT UNIZD)

26.10.2026.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

- 1 Autentikacija vs autorizacija - razlika i granica
- 2 Role: admin i club korisnik
- 3 Ownership: korisnik smije mijenjati samo svoje podatke
- 4 Dependency-ji za provjeru role i ownership
- 5 401 vs 403 - kad koji status code
- 6 Middleware vs dependency pristup
- 7 Testovi za autorizaciju

Na kraju predavanja: endpointi provjeravaju **tko si i smiješ li to raditi**.

Nakon ovog predavanja student može:

- objasniti razliku između autentikacije i autorizacije
- implementirati provjeru role u FastAPI dependencyju
- implementirati ownership provjeru u service layeru
- razlikovati 401 i 403 status code i objasniti kad se koji koristi
- napisati test koji provjerava da korisnik ne može pristupiti tuđim podacima

Modeli autorizacije: RBAC, ABAC, ACL

Model	Kako radi
ACL (Access Control List)	Svaki resurs ima listu tko smije što (npr. Unix file permissions)
RBAC (Role-Based)	Korisnik ima rolu, rola ima dozvole. Jednostavno za male sustave.
ABAC (Attribute-Based)	Odluka na temelju atributa korisnika, resursa i konteksta. Fleksibilno ali složeno.

Naš projekt koristi **RBAC + ownership** - dovoljno za domenu, a studenti razumiju model.

Zašto RBAC za naš projekt?

- Imamo samo 2 role (admin, club) - RBAC je prirodan izbor
- ABAC bi bio overkill: nemamo složene politike poput „klub smije prijaviti samo liftera starijeg od 18 godina iz istog grada”
- ACL ne skalira: ne želimo definirati pristup za svaki resurs pojedinačno

Kad bi trebao ABAC?

Kad imate 10+ rola, politike ovise o vremenu/lokaciji, ili pravilima koja se mijenjaju bez deploya.

OWASP Top 10: Broken Access Control (#1)

Od 2021., **Broken Access Control** je #1 ranjivost prema OWASP-u. OWASP je kratica za **Open Web Application Security Project** - neprofitna organizacija koja objavljuje liste najčešćih sigurnosnih propusta.

Tipični propusti:

- 1 Provjera pristupa samo na frontendu
- 2 Endpoint bez ikakve provjere role
- 3 Korisnik mijenja ID u URL-u i pristupa tuđim podacima
- 4 API vraća podatke koje korisnik ne bi smio vidjeti

Naš projekt pokriva sve navedeno: backend provjerava rolu i ownership na **svakom** endpointu.

IDOR: Insecure Direct Object Reference

Napadač mijenja ID u requestu da pristupi tuđim podacima.

Primjer napada:

- 1 Korisnik kluba Behemot šalje: `GET /clubs/1/lifters` (svoj klub)
- 2 Mijenja URL u: `GET /clubs/2/lifters` (klub Heraklo)
- 3 Bez ownership provjere: vidi tuđe natjecatelje

Obrana: usporedba `current_user.club_id` s `club_id` iz URL-a u service layeru.

IDOR je najčešći oblik Broken Access Controla.

Horizontalna vs vertikalna eskalacija

	Horizontalna	Vertikalna
Što	Pristup podacima drugog korisnika iste role	Pristup funkcijama više role
Primjer	Klub A vidi liftera kluba B	Club korisnik kreira natjecanje (admin akcija)
Obrana	Ownership provjera	Role provjera

Moramo se braniti od **obje** eskalacije. `require_role` brani od vertikalne, `ownership` od horizontalne.

Principle of Least Privilege

Korisnik smije **samo ono** što mu treba za posao. Ništa više.

U praksi:

- Club korisnik ne vidi API za kreiranje natjecanja - čak ni u Swagger dokumentaciji
- Admin ne modificira prijave klubova direktno - to je klubova odgovornost
- Endpointi koji vraćaju liste filtriraju prema roli: admin vidi sve, club vidi svoje

Defense in Depth: višeslojni pristup

Ne oslanjamo se na jednu provjeru:

- 1 **Transport:** HTTPS (enkriptirani promet)
- 2 **Autentikacija:** JWT token verifikacija
- 3 **Autorizacija - rola:** `require_role` dependency
- 4 **Autorizacija - ownership:** service layer provjera
- 5 **Baza:** foreign key i unique constrainti

Ako jedan sloj zakaže, problem zaustavlja sljedeći.

JWT claims: što stavljamo u token?

Naš JWT payload sadrži:

- sub - user ID
- role - "admin" ili "club"
- club_id - ID kluba (null za admina)
- exp - istek tokena

Pravilo: u token stavimo samo ono što nam treba za autorizaciju na **svakom** requestu.

Zašto ne čitamo rolu iz baze na svakom requestu?

Performanse: upit u bazu na svaki request je skup. JWT je self-contained - sve što treba je u tokenu. Trade-off: ako se rola promijeni, stari token i dalje ima staru rolu do isteka.

Token revokacija: trade-off

Problem: korisnik je deaktiviran, ali ima valjan token do isteka.

Opcije:

- ❶ **Kratki tokeni** (15 min) - kompromis brzine i sigurnosti
- ❷ **Blacklist** - lista revociranih tokena (zahtijeva storage, npr. Redis)
- ❸ **Verzioranje** - `token_version` u bazi, odbijaj stare verzije

Naš projekt koristi kratke tokene (15 min access + refresh). Na obrani morate znati objasniti zašto nismo implementirali blacklist.

Auto-login: kreiranje kluba s korisnikom

Kad admin kreira klub:

- 1 Admin šalje: `POST /clubs` s `name`, `city`, `password`
- 2 Backend kreira Club i automatski kreira User:
 - `username = slugify(name)` - npr. „Behemot” → `behemot`
 - `role = "club"`
 - `club_id = novi_klub.id`
- 3 Response vraća klub podatke + generirani username

Club 1:1 User - svaki klub ima točno jednog korisnika za prijavu.

Admin vidi username i proslijeđuje ga klubu. Klub se prijavi s username + password.

Kontekst: gdje smo u projektu?

- Predavanje 3: korisnik se može **prijaviti** i dobiti JWT token
- `get_current_user` dependency izvlači korisnika iz tokena
- Danas: dodajemo sloj koji provjerava **smiješ li raditi to što tražiš**

Autentikacija = tko si ti.
Autorizacija = smiješ li to.

Naš model autorizacije

Rola	Ovlasti
admin	Kreira natjecanja, kreira klubove i korisnike, vidi sve prijave, upravlja rokovima
club	Registrira natjecatelje svog kluba, prijavljuje ih na natjecanje, vidi samo svoje podatke

Rola je pohranjena u JWT payloadu i u bazi (`users.role`).

Ownership: zašto rola nije dovoljna?

- Dva korisnika imaju rolu club
- Korisnik kluba „Behemot” ne smije uređivati natjecatelje kluba „Heraklo”
- Rola kaže **što smiješ raditi** - ownership kaže **nad čijim podacima**

Ownership se provjerava usporedbom `current_user.club_id` s `club_id` resursa.

401 vs 403: kad koji?

Status	Značenje
401 Unauthorized	Nisi se identificirao ili ti je token istekao. Ponovno se prijavi.
403 Forbidden	Znam tko si, ali nemaš pravo na to. Ponovna prijava neće pomoći.

Česta greška: vratiti 401 kad bi trebao biti 403. To zbunjuje frontend koji pokušava refreshati token umjesto da prikaže „nemate pristup”.

Dependency: require_role

```
from app.core.errors import AppError

def require_role(*allowed_roles: str):
    def checker(
        current_user: User = Depends(get_current_user)
    ) -> User:
        if current_user.role not in allowed_roles:
            raise AppError(
                "forbidden",
                "Nemate dozvolu za ovu akciju",
                status_code=403,
            )
        return current_user
    return checker
```

Factory pattern: `require_role("admin")` vraća dependency koji propušta samo admina.

Korištenje require_role u routeru

```
@router.post("/")
async def create_club(
    body: ClubCreate,
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_role("admin")),
):
    # auto-kreira User s username=slugify(name)
    return await club_service.create_club(db, body)

# preview: ovo implementiramo na P5
@router.get("/{club_id}/lifters")
async def list_lifters(
    club_id: int,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(require_role("admin", "club")),
):
    return await lifter_service.list_by_club(db, club_id, user)
```

Gdje provjeravati ownership?

Sloj	Odgovornost
Router	Parsira request, poziva service. Ne provjerava ownership.
Service	Provjerava ownership i poslovna pravila. Baca <code>AppError</code> .
Repository	Radi DB upit. Ne zna za role ni ownership.

Ownership je **poslovno pravilo**, ne HTTP koncept. Zato živi u service layeru.

Service: ownership provjera (preview za P5)

```
async def list_lifters_by_club(
    db: AsyncSession,
    club_id: int,
    current_user: User,
) -> list[Lifter]:
    if current_user.role == "club" and current_user.club_id != club_id:
        raise AppError(
            "forbidden",
            "Ne mozete pristupiti natjecateljima drugog kluba",
            status_code=403,
        )
    return await lifter_repo.get_by_club(db, club_id)
```

Admin smije vidjeti sve. Club korisnik smije vidjeti samo svoj klub. Lifter endpoint implementiramo na P5, ovdje pokazujemo **pattern**.

Alternativa: ownership kao dependency

```
async def require_club_access(
    club_id: int,
    current_user: User = Depends(get_current_user),
) -> User:
    if current_user.role == "admin":
        return current_user
    if current_user.club_id != club_id:
        raise AppError("forbidden", "Pristup odbijen", 403)
    return current_user
```

Ovo radi ako je `club_id` uvijek u URL-u. Za složenije slučajeve ownership ostaje u serviceu.

Admin vidi sve, club vidi svoje

Tipični pattern u našem projektu (neki endpointi dolaze na P5):

Endpoint	Admin	Club	Provjerava
GET /clubs/{id}/lifters	svi	samo svoj klub	ownership
POST /competitions/{id}/registrations	sve	samo svoj klub	ownership
POST /competitions	da	ne	role
GET /clubs	svi	samo svoj	ownership
POST /clubs	da	ne	role

Česte greške u autorizaciji

- 1 Provjera role samo na frontendu - backend je autoritet
- 2 Ownership provjera samo po URL-u, ne po tijelu requesta
- 3 Zaboraviti provjeriti ownership na UPDATE/DELETE
- 4 Vratiti 404 umjesto 403 - otkriva postojanje resursa
- 5 Ne testirati autorizaciju uopće

Na obrani testiram točno ovo: šaljem request s tokenom korisnika koji **ne bi smio** imati pristup.

404 vs 403: dilema otkrivanja resursa

- Ako vratim 403: napadač zna da resurs postoji
- Ako vratim 404: napadač ne zna, ali korisnik ne razumije zašto ne vidi podatke

Uobičajena praksa:

- Za javne resurse (natjecanja): 403 je ok, ionako su javni
- Za osjetljive resurse: 404 je sigurnija opcija

U našem projektu koristimo 403 jer domena nije osjetljiva. Na obrani morate znati objasniti zašto.

Test: autorizacija po roli

```
def test_club_user_cannot_create_club(client, club_token):
    r = client.post(
        "/clubs",
        json={"name": "Novi", "city": "Zagreb", "password": "pass123"},
        headers={"Authorization": f"Bearer {club_token}"},
    )
    assert r.status_code == 403

def test_admin_can_create_club(client, admin_token):
    r = client.post(
        "/clubs",
        json={"name": "Novi", "city": "Zagreb", "password": "pass123"},
        headers={"Authorization": f"Bearer {admin_token}"},
    )
    assert r.status_code == 201
    assert r.json()["username"] == "novi"
```

Test: ownership

```
def test_club_cannot_see_other_clubs_lifters(
    client, club_a_token, club_b_id
):
    r = client.get(
        f"/clubs/{club_b_id}/lifters",
        headers={"Authorization": f"Bearer {club_a_token}"},
    )
    assert r.status_code == 403

def test_club_can_see_own_lifters(
    client, club_a_token, club_a_id
):
    r = client.get(
        f"/clubs/{club_a_id}/lifters",
        headers={"Authorization": f"Bearer {club_a_token}"},
    )
    assert r.status_code == 200
```

Testne fixture za auth tokene

Testovi trebaju tokene za razne korisnike. Fixture pristup:

- `admin_token` - fixture koja seedira admina i vrati token
- `club_a_token` - fixture za korisnika kluba A
- `club_b_token` - fixture za korisnika kluba B

Alternativa: `dependency_overrides` iz prošlog predavanja. Direktni tokeni su bolji jer testiraju **cijeli stack** uključujući JWT verifikaciju.

Middleware vs dependency za autorizaciju

	Middleware	Dependency
Kad se izvršava	Prije routera, na svaki request	Samo na endpointima koji ga koriste
Granularnost	Gruba (po prefixu URL-a)	Fina (po endpointu)
Pristup podacima	Samo request/response	Može primiti path parametre

FastAPI dependency pristup je bolji za naš slučaj:

- Različiti endpointi trebaju različite role
- `club_id` iz URL-a nam treba za ownership provjeru
- Middleware ne može znati koji je `club_id` relevantan

Audit checklist: jeste li sve pokrili?

Prije obrane provjerite svaki endpoint:

- 1 Zahtijeva li autentikaciju? (`get_current_user` ili `require_role`)
- 2 Provjerava li rolu? (admin-only vs club-allowed)
- 3 Provjerava li ownership? (club smije samo svoje)
- 4 Postoji li test za neautorizirani pristup?
- 5 Je li response konzistentan? (401 vs 403 na pravom mjestu)

Ako imate 5 endpointa, trebate barem 5 testova za autorizaciju - po jedan negativni za svaki endpoint.

Commitovi za Predavanje 4

- feat(api): add require_role dependency factory
- feat(api): add club schemas and repository
- feat(api): add admin club endpoints (create with auto-login, list, update, reset-password)
- feat(api): enforce ownership checks in service layer
- test(api): add role and ownership tests (admin/club/cross-club)

Na satu: prolazimo diffeve i testiramo sa Swaggerom - šaljemo requestove s raznim tokenima.

Što ide sljedeće (Predavanje 5)

- Modeliranje preostalih entiteta: Lifter, Competition, Registration
- CRUD operacije kroz sve slojeve
- Pydantic schema dizajn: Create vs Update vs Response
- Validacija inputa i smisleni error odgovori
- Paginacija i filtriranje

Pitanja?